

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 - Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	N. S. V. Trivikram	Reg. No.:	192525063
Section / Batch:	AIML / 2025	Date:	04/09/2026

Code of Conduct

I N. S. V. Trivikram (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 192525063.

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

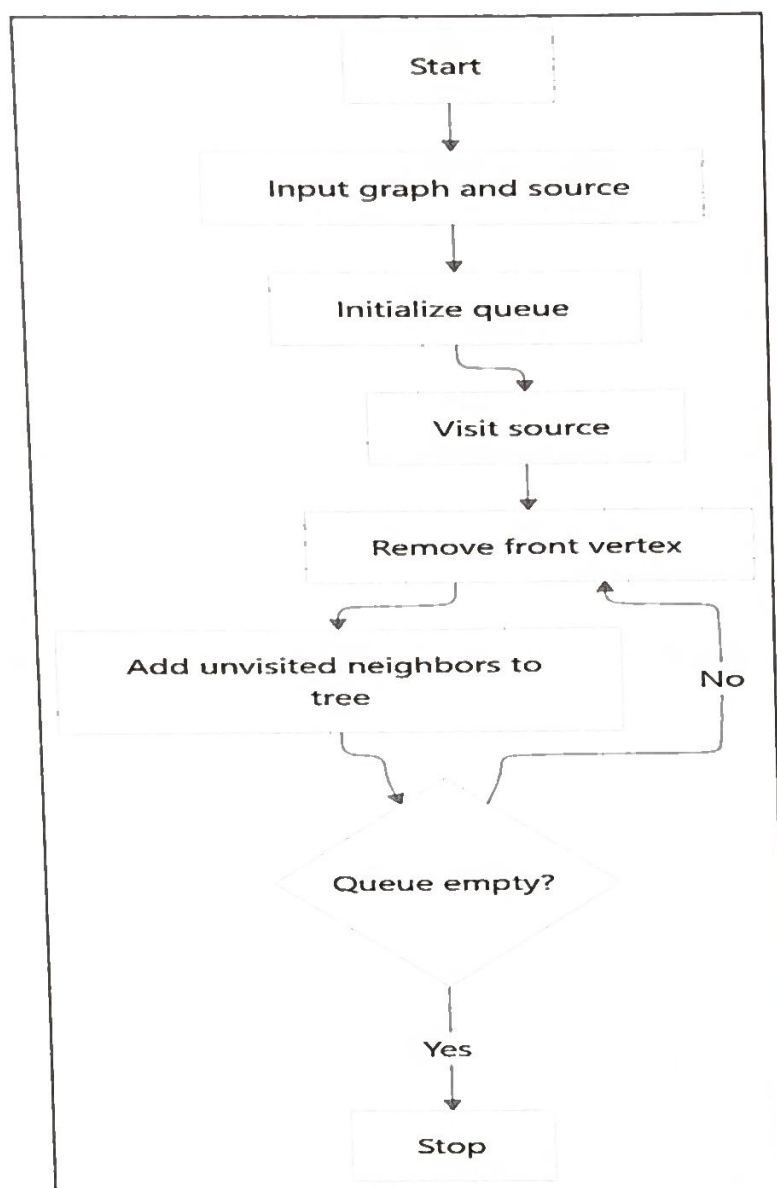
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Pseudocode Writing Task

1. Convert the flowchart for generating a BFS tree into code.

(20 Marks)

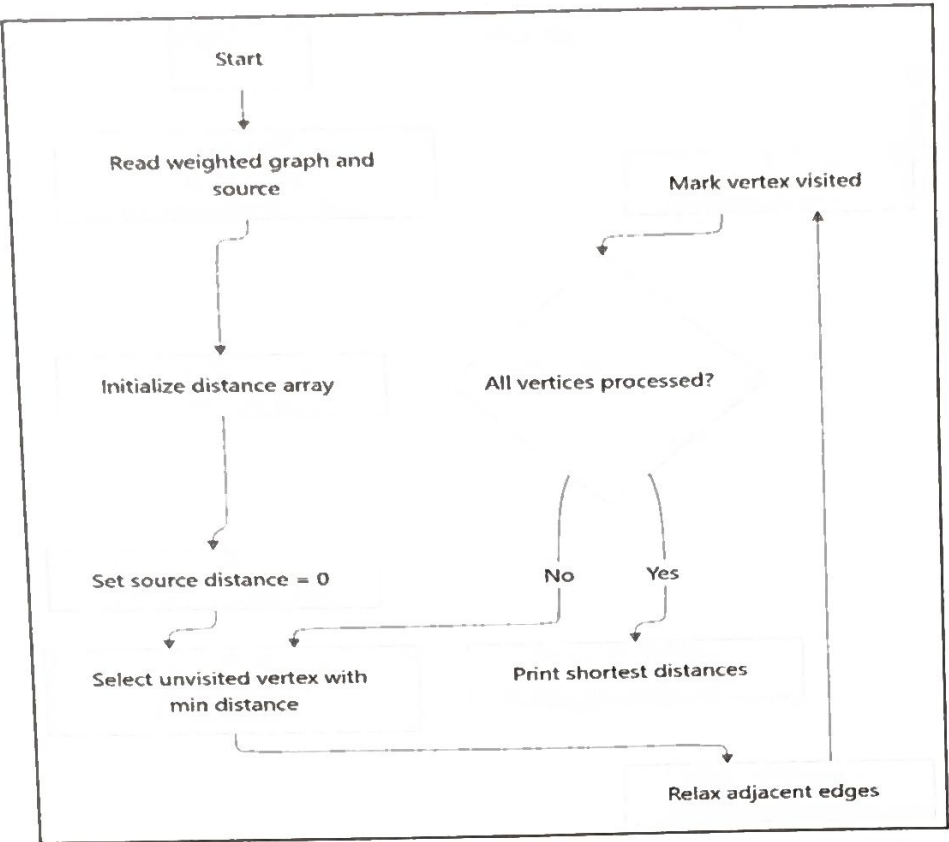
Flowchart Diagram:



2. Convert the flowchart for Dijkstra's shortest path algorithm into code.

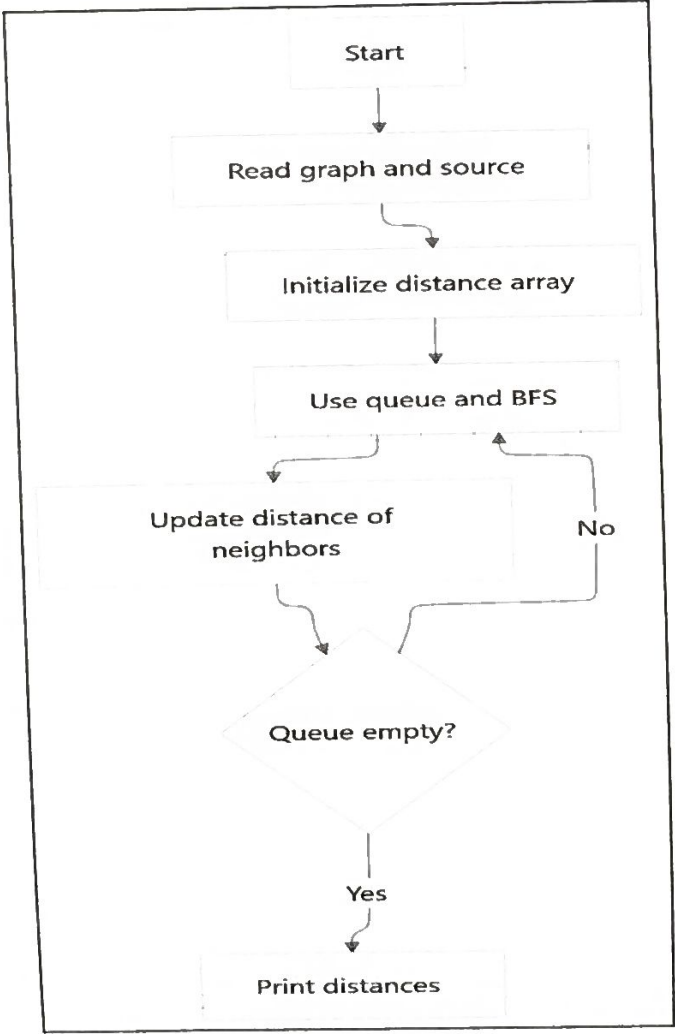
(20 Marks)

Flowchart Diagram:



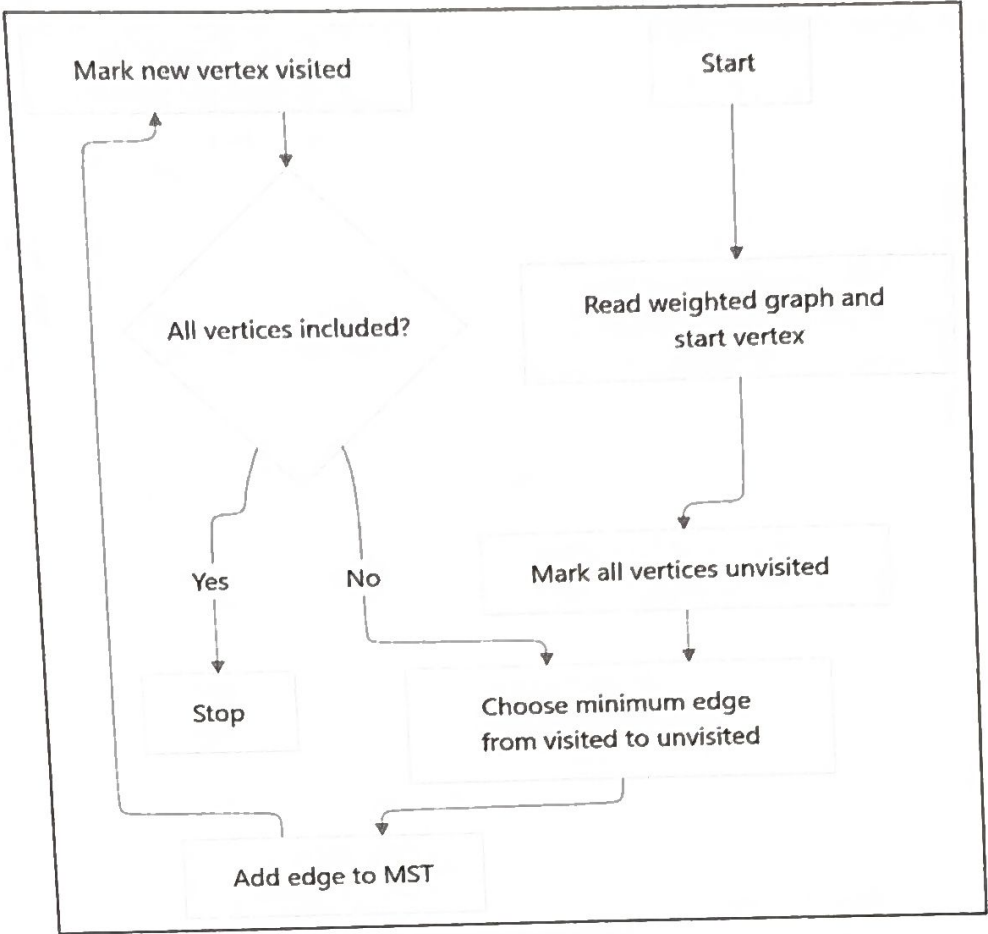
3. Convert the flowchart for shortest path in an unweighted graph into code. (20 Marks)

Flowchart Diagram:



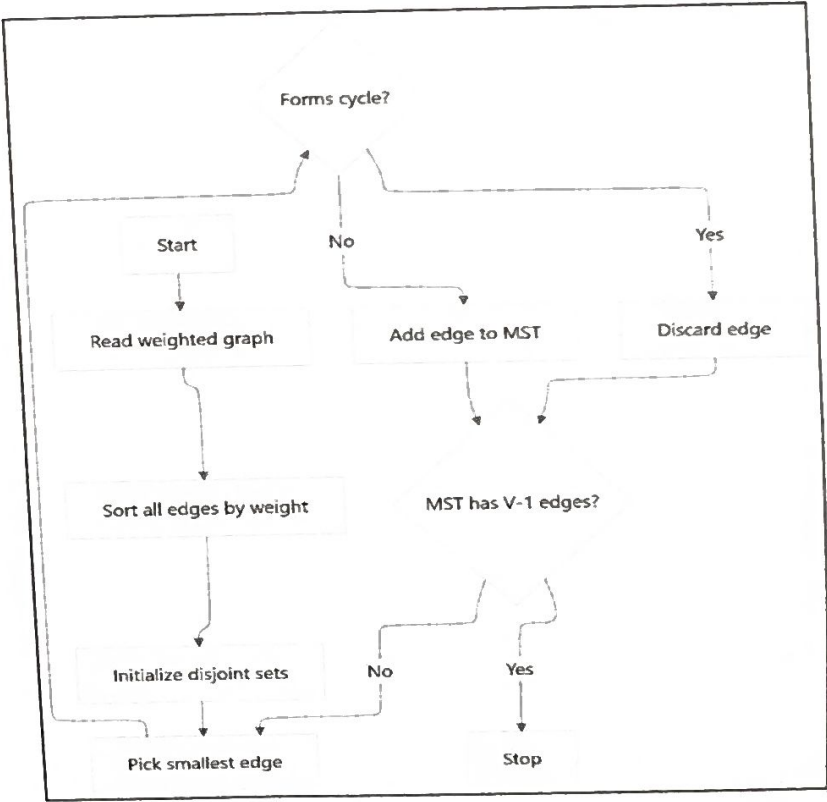
4. Convert the flowchart for Prim's algorithm into code. (20 Marks)

Flowchart Diagram:



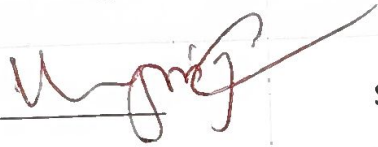
5. Convert the flowchart for Kruskal's algorithm into code. (20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

BFS Tree

```
include <stdio.h>
```

```
define MAX20
```

```
int graph[MAX][MAX];
```

```
int visited[MAX];
```

```
int queue[MAX];
```

```
int front = 0, rear = 0;
```

```
int n;
```

```
void BFS(int start)
```

```
{
```

```
    visited[start] = 1;
```

```
    queue[rear++] = start;
```

```
    Print("BFS Traversal:");
```

```
    while (front < rear)
```

```
    {
```

```
        int current = queue[front++];
```

```
        Printf("%d", current);
```

```
        for (int i = 0; i < n; i++)
```

```
        {
```

```
            if (graph[current][i] == 1 && visited[i] == 0)
```

```
            {
```

```
                visited[i] = 1;
```

```
                queue[rear++] = i;
```

```
                Printf("\n BFS Tree Edge: %d -> %d",
```

```
                    current, i);
```

```
            }
```

```
        }
```

```
    }
```

```
    Printf("\n");
```

```
}
```

```
int main()
```

{

int start;

printf("Enter number of vertices:");

scanf("%d", &n);

printf("Enter adjacency matrix: (n)");

for (int i = 0; i < n; i++)

{

for (int j = 0; j < n; j++)

{

scanf("%d", &graph[i][j]);

}

}

printf("Enter starting vertex:");

scanf("%d", &start);

BFS(start);

return 0;

}

Dijkstra's Shortest Path Algorithm

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 20
```

```
int graph[MAX][MAX];
```

```
int distance[MAX];
```

```
int visited[MAX];
```

```
int n;
```

```
int findMinVertex()
```

```
{
```

```
    int min = INT_MAX;
```

```
    int minVertex = -1;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        if (visited[i] == 0 && distance[i] < min)
```

```
        {
```

```
            min = distance[i];
```

```
            minVertex = i;
```

```
        }
```

```
    }
```

```
    return minVertex;
```

```
}
```

```
void dijkstra(int source)
```

```
{
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        distance[i] = INT_MAX;
```

```
        visited[i] = 0;
```

```
    }
```

```
    distance[source] = 0;
```

```
    for (int count = 0; count < n - 1; count++)
```

```
    {
```

```
        int v = findMinVertex();
```



```
if (v == -1)
```

```
break;
```

```
visited[v] = 1;
```

```
for (int v=0; v<n; v++)
```

```
{
```

```
if (visited[v] == 0 &&
```

```
graph[u][v] != 0 &&
```

```
distance[u] != INT_MAX &&
```

```
distance[u] + graph[u][v] < distance[v])
```

```
{
```

```
distance[v] = distance[u] + graph[u][v];
```

```
}
```

```
}
```

```
printf("\n shortest distance from vertex %d:\n", source);
```

```
for (int i=0; i<n; i++)
```

```
{
```

```
if (distance[i] == INT_MAX)
```

```
{
```

```
printf("vertex %d: INF\n", i);
```

```
}
```

```
else
```

```
{
```

```
printf("vertex %d: %d\n",
```

```
i, distance[i]);
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
int source;
```

```
printf("Enter number of vertices:");
```

```
printf("Enter weighted adjacency matrix:\n");
```

```
printf("Enter source vertex:");
```

```
scanf("%d", &source);
```

```
dijkstra(source);
```

```
return 0;
```

```
}
```

Unweighted Graph

include <stdio.h>

define MAX 20

int graph[MAX][MAX];

int visited[MAX];

int visited[MAX];

int Parent[MAX];

int queue[MAX];

int n;

int front = 0;

int rear = 0;

void BFS (int source)

{
for (int i = 0; i < n; i++)

{
visited[i] = 0;

distance[i] = -1;

Parent[i] = -1;

}

visited[source] = 1;

distance[source] = 0;

queue[rear++] = source;

while (front < rear)

{

int current = queue[front++];

for (int i = 0; i < n; i++)

{

if (graph[current][i] == 1 &&

visited[i] == 0)

{

visited[i] = 1;

distance[i] = distance[current] + 1;

Parent[i] = current;

```
queue [rear++] = i;
```

```
}  
}  
}
```

```
void printPath (int vertex)
```

```
{
```

```
if (vertex == -1)
```

```
return;
```

```
PrintPath (parent [vertex]);
```

```
Printf ("%d", vertex);
```

```
}
```

```
int main ()
```

```
{
```

```
int source;
```

```
int destination;
```

```
Printf ("Enter number of vertices:");
```

```
scanf ("%d", &n);
```

```
Printf ("Enter adjacency matrix:\n");
```

```
for (int i=0; i<n; i++)
```

```
{
```

```
for (int j=0; j<n; j++)
```

```
{
```

```
scanf ("%d", &graph[i][j]);
```

```
}
```

```
Printf ("Enter source vertex:");
```

```
scanf ("%d", &source);
```

```
BFS(source);
```

```
Printf ("Enter destination vertex:");
```

```
scanf ("%d", &destination);
```

```
if (distance [destination] == -1)
```

```
{
```

```
Print ("No path exists from %d to %d.\n",  
source, destination);
```

```
}
```

4. Prim's Algorithm

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 20
```

```
int graph[MAX][MAX];
```

```
int selected[MAX];
```

```
int n;
```

```
void prim ( )
```

```
{
```

```
    int edgeCount = 0;
```

```
    int totalWeight = 0;
```

```
    selected[0] = 1;
```

```
    printf ("In Edges in Minimum Spanning Tree : (n)");
```

```
    while (edgeCount < n-1)
```

```
    {
```

```
        int min = INT_MAX;
```

```
        int x = -1;
```

```
        int y = -1;
```

```
        for (int i = 0; i < n; i++)
```

```
        {
```

```
            if (selected[i] == 1)
```

```
            {
```

```
                for (int j = 0; j < n; j++)
```

```
                {
```

```
                    if (selected[j] == 0 &&
```

```
                        graph[i][j] != 0 &&
```

```
                            graph[i][j] < min)
```

```
                    {
```

```
                        min = graph[i][j];
```

```
                        x = i;
```

```
                        y = j;
```

```
                    }  
                }  
            }  
        }
```



```
if (x == -1 || y == -1)
```

```
{  
    printf("Graph is disconnected\n");  
    return;  
}
```

```
printf("%d %d : %d\n", x, y, min);
```

```
totalWeight += min;
```

```
selected[v] = 1;
```

```
edge count ++;
```

```
}  
printf("\n Total MST weight = %d\n", totalWeight);
```

```
{
```

```
int main ( )
```

```
{
```

```
printf("Enter number of vertices : ");
```

```
scanf("%d", &n);
```

```
printf("Enter weighted adjacency matrix : \n");
```

```
for (int i=0; i<n; i++)
```

```
{  
    for (int j=0; j<n; j++)
```

```
{  
    scanf("%d", &graph[i][j]);
```

```
}
```

```
}
```

```
prim();
```

```
return 0;
```

```
}
```

5. kruskal's algorithm

```
#include <stdio.h>
```

```
#define MAX 100
```

```
typedef struct
```

```
{
```

```
    int source;
```

```
    int destination;
```

```
    int weight;
```

```
} Edge;
```

```
Edge edges [MAX];
```

```
int parent [MAX];
```

```
int n;
```

```
int edgeCount;
```

```
int find (int vertex)
```

```
{
```

```
    if (parent[vertex] == vertex)
```

```
        return vertex;
```

```
    return parent[vertex] = find (parent[vertex]);
```

```
}
```

```
void unionSet (int a, int b)
```

```
{
```

```
    int rootA = find(a);
```

```
    int rootB = find(b);
```

```
    parent parent[rootB] = rootA;
```

```
}
```

```
void sortEdges ()
```

```
{
```

```
    for (int i = 0; i < edgeCount - 1; i++)
```

```
    {
```

```
        for (int j = 0; j < edgeCount - i - 1; j++)
```

```
        {
```

```
            if (edges[j].weight > edges[j+1].weight)
```

```
            {
```

```
                Edge temp = edges[j];
```

```

edges[i] = edges[i+1];
edges[i+1] = temp;
}
}
}

```

```

void kruskal ( )
{

```

```

    int mstEdges = 0;

```

```

    int totalWeight = 0;

```

```

    sort(edges);

```

```

    for (int i = 0; i < n; i++)
    {

```

```

        parent[i] = i;
    }

```

```

    printf("\nEdges in Minimum Spanning Tree:\n");

```

```

    for (int i = 0;

```

```

        i < edgeCount && mstEdges < n-1;
        i++)
    {

```

```

        int u = edges[i].source;

```

```

        int v = edges[i].destination;

```

```

        int rootU = find(u);

```

```

        int rootV = find(v);

```

```

        if (rootU != rootV)
        {

```

```

            printf("%d - %d : %d\n",

```

```

                u, v, edges[i].weight);

```

```

            totalWeight += edges[i].weight;

```

```

            unionSet (rootU, rootV);

```

```

            mstEdges++;
        }
    }
}

```

if (numEdges != n-1)

{ printf("Graph is disconnected.\n");

}

else

{ printf("Total num weight = %.d\n",
totalWeight);

}

}

int main()

{

printf("Enter number of vertices:");

scanf("%d", &n);

printf("Enter number of edges:");

scanf("%d", &edgeCount);

printf("Enter edges (source destination weight): \n");

for (int i=0; i < edgeCount; i++)

{

scanf("%d %d %d",

&edges[i].source,

&edges[i].destination,

&edges[i].weight);

}

krukal();

return 0;

Woj